



Writeup

CURSO:
SEGURIDAD DE SOFTWARE

RAINING PRIMES (HARD)

Profesor:
Natán Fernández de Castro

Estudiantes
Kener Adiel Castillo
Alejandro Cerdas Vargas

Carnet
2023179846
2023073037

23 de junio de 2026

Índice

1. Información de Contacto	2
2. Resumen ejecutivo	2
2.1. Alcance	2
2.2. Hallazgos y recomendaciones	2
3. Información general	2
4. Reconocimiento	3
4.1. Recopilación inicial de información	3
4.2. Elementos identificados	4
5. Enumeración y análisis	6
5.1. Hallazgos	6
5.2. Posibles vectores de ataque	10
5.3. Caminos probados	10
6. Explotación	14
6.1. Vulnerabilidades identificadas	14
6.2. Pasos para explotación	16
6.2.1. Paso 1	16
6.2.2. Paso 2	19
6.2.3. Paso 3	20
6.2.4. Paso 4	21
6.2.5. Paso 5	21
6.2.6. Implementación	21
6.2.7. Demostración	27
6.2.8. Aspectos adicionales	29
7. Evidencia de resolución	30
8. Remediación	31
9. Lecciones aprendidas	32
10. Acrónimos	32
11. Recursos consultados	33

1. Información de Contacto

Nombre	Correo Electrónico	Número de Teléfono
Alejandro Cerdas Vargas	al.cerdas@estudiantec.cr	506-8403-4568
Kener Adiel Castillo	kencastillo@estudiantec.cr	506-7020-8452

Cuadro 1: Información de los testers

2. Resumen ejecutivo

2.1. Alcance

Este challenge de dificultad difícil consistió en encontrar las vulnerabilidades que tenía un script de Python que implementaba el algoritmo de encriptación Rivest, Shamir y Adleman ([RSA](#)) junto con Advanced Encryption Standard ([AES](#)) para cifrar un mensaje. El objetivo consistió en recuperar la clave privada utilizada por el algoritmo [RSA](#) para proteger el mensaje cifrado con [AES](#). Una vez logrado esto último, se buscó forzar la generación de una clave [AES](#) predecible, de modo que fuera posible obtenerla y, posteriormente, descifrar el mensaje cifrado con [AES](#).

2.2. Hallazgos y recomendaciones

En este challenge, se encontraron dos vulnerabilidades altamente explotables, una en la implementación del algoritmo [AES](#) y otra en el [RSA](#). La primera vulnerabilidad consiste en la posibilidad de generar una llave para [AES](#) predecible al momento de cambiar la llave anterior. La segunda vulnerabilidad, presenta la posibilidad de descifrar aquellos textos protegidos con [RSA](#), debido a que se puede obtener la llave privada de [RSA](#) mediante un análisis numérico de la forma en que se generan las claves. Ambas vulnerabilidades representan graves problemas para la seguridad de cualquier sistema.

Las principales recomendaciones en este caso consisten en elegir una clave segura y aleatoria para el algoritmo [AES-256](#), que no permita modificación externa. De igual forma, para el algoritmo [RSA](#) es necesario asegurarse de que los números usados para la generación de las claves sean completamente aleatorios e independientes, usando un método de generación seguro que no introduzca efectos o relaciones matemáticas que comprometan la seguridad del programa. También es importante mantener tamaños de números adecuados (mayor o igual a 1024 bits) para evitar explotaciones. Por último, se recomienda mantener las prácticas relacionadas con los principios de seguridad desde el diseño, como el uso de estándares criptográficos correctos y requerir el uso de herramientas de seguridad comprobada.

3. Información general

El challenge resuelto para este write-up pertenece a la plataforma Hack The Box ([HTB](#)). Más detalles en la tabla [2](#).

Task	Raining Primes
Dificultad	Hard
Categoria	Cripto
Fecha de Resolución	4 de junio del 2026
Usuario	aleCaleCaleC y kencast

Cuadro 2: Información General

4. Reconocimiento

4.1. Recopilación inicial de información

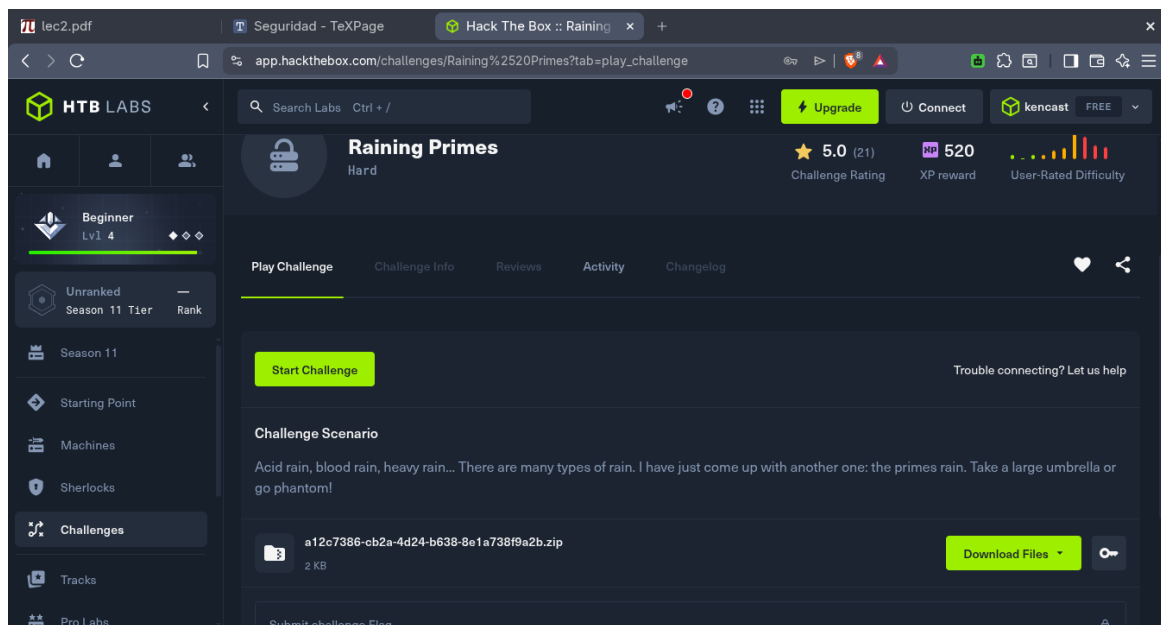


Figura 1: El challenge en HTB

Para iniciar el challenge hay que descargar el archivo `a12c7386-cb2a-4d24-b638-8e1a738f9a2b.zip` en HTB, como se muestra en la figura 1.

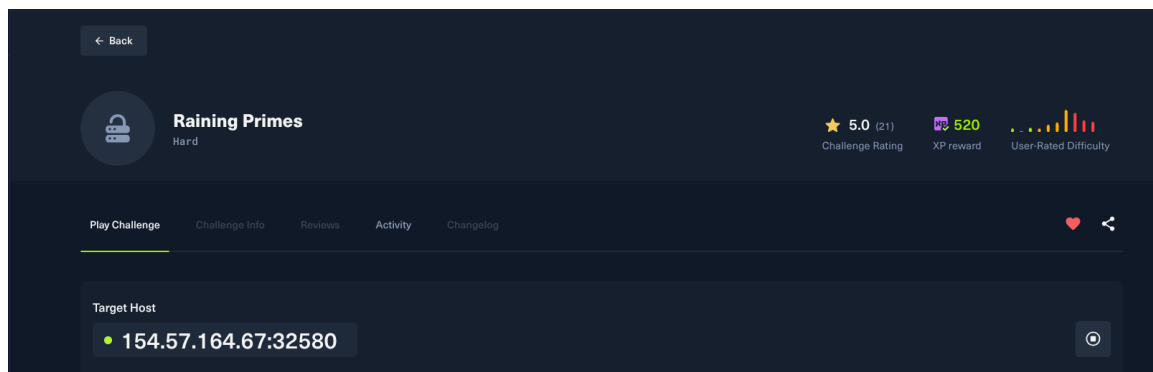


Figura 2: Inicio de la máquina remota

Una vez descargado el archivo, hay que darle al botón de **Start Challenge**, lo cual inicializa una máquina remota y nos da la IP para conectarnos con OpenVPN, como se observa en la figura 2.

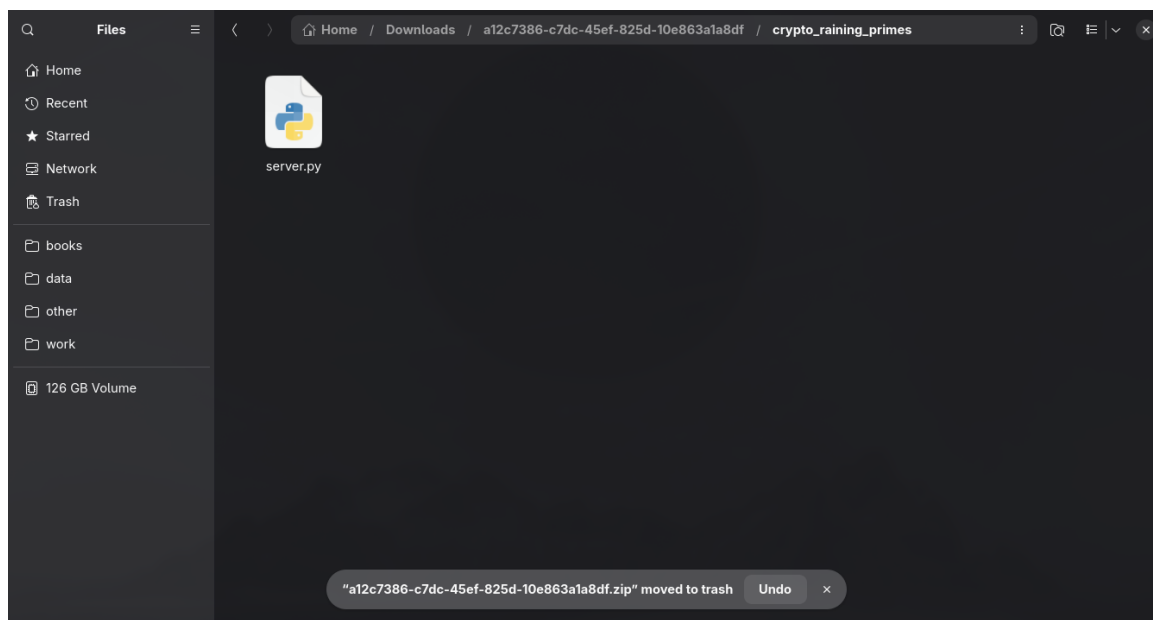


Figura 3: Archivos del challenge

Después, hay que descomprimir el archivo zip, en HTB se encuentra la contraseña. En total se cuenta con solo un archivo, que corresponde con un script del lenguaje Python `server.py`, como se observa en la figura 3. Este archivo corresponde al código que se ejecuta en la máquina remota, y al tratarse de un challenge criptográfico, esto sugiere que hay que analizar el código de Python en busca de vulnerabilidades.

4.2. Elementos identificados

A pesar que se poseía una IP, no se utilizaron herramientas de red ni se llevaron a cabo estudios sobre la misma, debido a que, el challenge es de categoría crypto, por lo cual el objetivo probablemente es descifrar una bandera, y no es de mucha utilidad realizar un análisis como el antes mencionado. Solo

se cuenta con el script y los resultados de la interacción con este, así que los elementos identificados son los siguientes:

Elemento	Tipo	Descripción
getPrime	Función	Pertenece al módulo de Python Crypto.Util.number. Retorna un número primo random de N bits. [2]
isPrime	Función	Pertenece al módulo de Python Crypto.Util.number. Verifica si el número dado es primo. [2]
RSA	Algoritmo	Método de encriptación asimétrica muy usado [1].
randbits	Función	Genera números enteros aleatorios de K bits [14]
AES	Clase	Implementa el cifrado simétrico por bloque [5]
pad	Función	Aplica “padding” estándar [6]
literal_eval	Función	Del módulo Ast, evalúa y devuelve una expresión de Python válida a partir del string dado.
to_bytes	Función	Función que convierte un número en bytes, ocupando cierta cantidad de bytes [15]
from_bytes	Función	Función que convierte bytes a un número [25]

Cuadro 3: Elementos identificados en los archivos

Las bibliotecas que usan parecen ser seguras y aptas para procesos criptográficos, por lo que estas no son el enfoque. El script representa más que todo un servidor que se ejecuta en la máquina remota con las funciones de:

- Generar primos de una manera particular.
- Cambiar la llave del algoritmo AES-256 en base a transformaciones del input.
- Cifrar el flag aplicando primero el algoritmo AES-256 y después RSA-2048. Imprime llave pública de RSA y el texto cifrado.

En [3] hay una explicación más detallada del algoritmo RSA, pero en general se resume en las fases de:

Generación de llaves:

Se eligen dos números primos grandes p y q , estos son secretos. Se calcula $n = pq$ como parte de la llave pública y privada. También, $\phi(n) = (p - 1)(q - 1)$ ya que p y q son números primos. Se elige $e : 1 < e < \phi(n) : \gcd(e, \phi(n)) = 1$, de esta manera son coprimos. Finalmente, se calcula un d tal que $d * e \equiv 1 \pmod{\phi(n)}$. La llave pública se representa por (n, e) y la privada por (n, d) .

Cifrado:

Si M es el mensaje representado de forma numérica. $C = M^e \bmod n$ es el mensaje cifrado.

Descifrado:

El mensaje original es $M = C^d \bmod n$.

A su vez el algoritmo de cifrado [AES](#), es un derivado del algoritmo Rijndael. [AES](#) opera bloques de 16 bytes en una matriz de 4×4 bytes. A esta matriz se le aplica cuatro operaciones básicas:

- **SubBytes** realiza una sustitución de cada byte de la matriz por uno correspondiente de una tabla fija.
- **ShiftRows** cada fila le realiza una rotación un número de veces.
- **MixColumns** cada columna la multiplica por un polinomio constante con el fin de realizarle una transformación lineal.
- **AddRoundKey** cada byte de la matriz se opera con un byte de una subclave, mediante el XOR.

Este procesos se lleva a cabo durante varios “rounds” [4].

5. Enumeración y análisis

5.1. Hallazgos

```
1  #!/usr/bin/env python3
2
3  from ast import literal_eval
4  from secrets import randbits
5
6  from Crypto.Cipher import AES
7  from Crypto.Util.number import getPrime, isPrime
8  from Crypto.Util.Padding import pad
9
10 A, B, P, R = 384, 256, 1024, 640
11
12 def a() -> int: return randbits(A) 2+ usages
13 def b() -> int: return randbits(B) 3+ usages
14
15 def n2v(n: int, k: int = B) -> list[int]: return list(map(int, f'{n:0{k}b}')) 1usage
16 def v2n(v: list[int]) -> int: return int(''.join(map(str, v)), 2) 1usage
17 def n2b(n: int, k: int = B // 8) -> bytes: return n.to_bytes(k, byteorder: 'big') 2+ usages
18 def b2n(b: bytes) -> int: return int.from_bytes(b, byteorder: 'big') 2+ usages
19 def v2b(v: list[int]) -> bytes: return n2b(v2n(v)) 1usage
20 def b2v(b: bytes) -> list[int]: return n2v(b2n(b)) 1usage
```

Figura 4: Imports y función de uso general del archivo `server.py`

En la figura 4, se pueden visualizar los imports de la mayoría de funciones presentadas en la tabla 4.2 (líneas 3-8). Además, se definen variables globales A,B,P y R, las cuales representan tamaños de ciertos números (línea 10).

Nombre	Utilidad
$a()$	Retorna un número aleatorio de 384 bits (línea 12).
$b()$	Retorna un número aleatorio de 256 bits (línea 13).
$n2v(n, k = 256)$	Retorna una lista con los bits del número n aplicando un padding de 256 (línea 15).
$v2n(v)$	Retorna el número decimal representado por la lista binaria de entrada (línea 16).
$n2b(n, k = 32)$	Retorna un total de 32 bytes, los cuales representan el número en “big-endian” (línea 17).
$b2n(b)$	Retorna el número decimal a partir de la representación en bytes de este (línea 18).
$v2b(v)$	Retorna los bytes del número representado en la lista (línea 19).
$b2v(b)$	Retorna una lista con el binario del número representado en los bytes (línea 20).

Cuadro 4: Explicación de las funciones presentadas en las líneas de 12-20 de la figura 4

```

23     class Server: 1usage
24         def __init__(self):
25             self.r = getPrime(R)
26             p, q = self.gen_prime(), self.gen_prime()
27             self.n, self.e = p * q, 65537
28             self.key = n2b(b())
29
30         def gen_prime(self) -> int: 3+ usages
31             while True:
32                 if (p := a() * self.r + b()).bit_length() == P and isPrime(p):
33                     return p
34
35         def _public_key(self, private_key: list[int]) -> list[int]: 1usage
36             m = [2 * b() + int(b_i) for b_i in private_key]
37             return [a() * self.r + m_i for m_i in m]
38
39         def update_key(self, k1: list[int]): 1usage
40             k0 = self._public_key(b2v(self.key))
41             assert isinstance(k1, list) and all(isinstance(k1_i, int) for k1_i in k1)
42             assert len(k0) == len(k1)
43             assert all(self.r < k1_i <= 2 ** A * self.r for k1_i in k1)
44             k = [k0_i * k1_i for k0_i, k1_i in zip(k0, k1)]
45             bits = [(k_i % self.r) % 2 for k_i in k]
46             self.key = v2b(bits)

```

Figura 5: Código de la clase **Server** del archivo `server.py`

En la figura 5, se define la clase **Server** con su constructor, el cual genera un primo r de 640 bits, genera dos primos p y q ; y apartir del producto de los mismos, calcula n . A su vez establece e igual a 65537 y key como los bytes de un número aleatorio de 256 bits (líneas 23-28).

La función `gen_prime` utilizada para generar p y q , genera dos números aleatorios, a de 384 bits y b de 256, y establece $p = a \times r + b$ (r definido en el constructor). Si este p es primo, lo retorna. De lo contrario, continúa el proceso hasta encontrar un p que lo cumpla (líneas 30-33). Realizando algunos intentos, se observó que en promedio la función realiza cerca de dos mil iteraciones.

La función `update_key` lo que hace es verificar que `k1` sea una lista de números de tamaño 256, mismo tamaño que la llave `self.key` del `AES`, y para $i \in [0, 255]$ que se cumpla $r < k_1[i] \leq 2^{384} * r$. Se define $k[i] = (a()r + 2b() + key[i]) * k_1[i]$ y se actualiza la llave como $key[i] = (k[i] \% r) \% 2$. Como a cada posición se le saca módulo dos, si el valor es par se vuelve cero, por lo que tal vez es posible elegir ciertos valores para que la llave del `AES` sea cero.

```

48         def _aes_encrypt(self, pt: bytes) -> bytes: 1usage
49             cipher = AES.new(self.key, AES.MODE_ECB)
50             return cipher.encrypt(pad(pt, AES.block_size))
51
52         def encrypt_flag(self) -> tuple[int, int, int]: 1usage
53             try:
54                 with open('flag.txt', 'rb') as f:
55                     flag = f.read().strip()
56             except FileNotFoundError:
57                 flag = b'HTB{f4k3_fl4g_f0r_t3st1ng}'
58
59             enc_flag = self._aes_encrypt(flag)
60             m = b2n(enc_flag)
61             c = pow(m, self.e, self.n)
62             return self.n, self.e, c

```

Figura 6: Código con las funciones de cifrado del archivo `server.py`

En la figura 6, se muestra un fragmento de código que todavía pertenece a la clase `Server` definida en la figura 5. En la línea 48, se define la función `_aes_encrypt` que recibe unos bytes, crea una instancia de `AES` (línea 49) con la llave definida en la figura 6 y se cifran dichos bytes (línea 50). Después en la línea 52, se define la función `encrypt_flag`, la cual intenta abrir un archivo llamado `flag.txt` y guarda su contenido en la variable `flag` (líneas 53-55). En caso de no poder abrir el archivo, se alambra `flag` con los bytes `HTB{f4k3_fl4g_f0r_t3st1ng}` (líneas 56 y 57). En la línea 59, se encripta `flag` con `AES` y se transforma a bytes (líneas 59 y 60). Finalmente, se cifra con `RSA` y se retornan `n`, `e` y el mensaje `c`.

```

64
65     def menu() -> int: 1usage
66         print('Choose an option:')
67         print('1. Generate prime')
68         print('2. Update key')
69         print('3. Encrypt flag')
70         print('4. Exit')
71         try:
72             return int(input('> '))
73         except ValueError:
74             return 4
75
76
77 ▶ if __name__ == '__main__':
78     server = Server()
79
80     while (option := menu()) != 4:
81         if option == 1:
82             print("It's raining primes!")
83             print(server.gen_prime())
84         elif option == 2:
85             try:
86                 c1 = literal_eval(input('Encrypted key:\n'))
87                 server.update_key(c1)
88             except AssertionError:
89                 break
90         elif option == 3:
91             print('Encrypted flag:')
92             print(server.encrypt_flag())
93

```

Figura 7: Código con el menú para la interacción del archivo `server.py`

En la figura 7, se muestra la función `menu`, la cual despliega el menú y lee la entrada (líneas 65-74). En la línea 77, se define que el fragmento de código de las líneas 78 a la 92 únicamente serán ejecutados si este archivo es compilado directamente. Además, se define la interacción con el menú.

Dato	Función
Llave pública de RSA	Encriptar la llave del AES . Hay que encontrar la llave privada o factorizar la llave pública para romper RSA .
Flag cifrado con AES y RSA	Descifrar el flag.
Generar primos	Obtener primos que comparten un mismo r .
Llave AES	Usuario puede influir sobre la definición de la llave AES , se puede forzar a un valor específico.

Cuadro 5: Datos que se tienen

En general, la implementación y los métodos usados parecen robustos, por lo que lo único que llama la atención es:

- Los números primos se generan de una forma particular, por lo que todos están relacionados por un valor r que es un número primo de 640 bits. Esto puede ser que introduzca una vulnerabilidad que permita factorizar la llave pública de [RSA](#).
- Se puede modificar la llave del algoritmo [AES](#), por lo que se puede forzar a cierto valor.

5.2. Posibles vectores de ataque

En [HTB](#), la bandera esperada corresponde a un texto de la forma `HTB{ }` con cierto contenido entre ambas llaves, y tomando en cuenta la información que se tiene, el objetivo es descifrar el mensaje. Con la información antes recaudada, se sabe que se tiene el módulo y el exponente usados para encriptar el mensaje ya cifrado con [AES](#) y, adicionalmente, se tiene el resultado tras la acción del [RSA](#).

Un vector de ataque contra [AES](#) en esta ocasión, es el hecho de que el usuario puede influir en la definición de la llave para el cifrado [AES](#) en la función `update_key` figura 5, ingresando un arreglo de 256 elementos que cumplen una serie de condiciones, interactuando con la opción del menú 2. Pero en este challenge, no basta con un único vector de ataque, ya que es necesario a su vez vulnerar [RSA](#). El vector de ataque propuesto para [RSA](#) es la factorización de la llave pública, ya que ambos primos que la conforman, comparten un primo en común, aprovechando las propiedades derivadas de la forma en que se generaron.

5.3. Caminos probados

Como antes se mencionó, este challenge se divide en dos partes: vulnerar [AES](#) y vulnerar [RSA](#); por lo tanto, los caminos intentados son independientes para ambos métodos. Para [AES](#), primero se tuvo la idea de que se podría hacer la llave de cifrado predecible convirtiéndola en 0. Aspecto posible si se lograba enviar un arreglo que cumpliera las condiciones descritas en la figura 5, y además fuera capaz de asegurar que el modulo 2 aplicado al final, fuese 0 para todas las posiciones. Analizando el código de la función `update_key()` de la figura 5, se puede observar

$$k_{n,i} = (k_{0,i} \times k_{1,i} (r)) \quad (1)$$

donde $k_{0,i}$ es la i -ésima posición del arreglo generado por la función `_public_key()`, $k_{1,i}$ la i -ésima posición del arreglo otorgado por el usuario y $k_{n,i}$ la i -ésima posición del arreglo que representa la llave final. A su vez, k_0 se puede representar como

$$k_{0,i} = a_i \times r + (2 \times b_i + k_{a,i}) \quad (2)$$

con $k_{a,i}$ siendo la i -ésima posición de la antigua llave. Reemplazando en la ecuación 1,

$$k_{n,i} = ((a_i \times r + 2 \times b_i + k_{a,i}) \times k_{1,i} (r)) \quad (2) \quad (3)$$

dejando de lado el módulo 2

$$k_{n,i} = (0 + 2 \times b_i + k_{a,i}) \times k_{1,i} (r) \quad (4)$$

Llegado a este punto se pensó en como se podía asignar un k_1 que hiciese la llave 0. Se puede usar la función `gen_prime()`, que genera un primo $\rho = a \times r + b$, el cual puede usarse para generar el arreglo de tamaño 256. Al remplazar

$$k_{n,i} = (0 + 2 \times b_i + k_{a,i}) \times (a_\rho \times r + b_\rho) (r) \quad (5)$$

$$k_{n,i} = (0 + 2 \times b_i + k_{a,i}) \times (0 + b_\rho) (r) \quad (6)$$

Se puede observar que ambos b son de aproximadamente 256 y k_a es 0 o 1, por lo cual su producto es inferior a r , que es de 640 bits, manteniendo la ecuación actual. Ahora, al aplicar el módulo dos

$$k_{n,i} = (0 + 0 + k_{a,i}) \times (0 + b_\rho) \quad (2) \quad (7)$$

por lo tanto, la llave queda como 0 únicamente cuando se satisface la condición

$$2 \mid k_{a,i} \vee 2 \mid b_\rho, \quad (8)$$

Se puede intuir que no se tiene control sobre el valor de $k_{a,i}$. Técnicamente, tampoco se tiene poder sobre ρ , el cual también debe cumplir la condición (figura 5)

$$r < \rho \leq r \times 2^{384} \quad (9)$$

Según [14] la función `randbits()` no necesariamente prende el bit más significativo, a diferencia de funciones como `getPrime()` (función que inicializa el `r`). Por lo tanto, algunos primos generados pueden cumplir esa condición de la ecuación 9, específicamente, si $a_\rho < 2^{384}$, ρ cumple dicha condición. De la misma forma, se sabe que b_ρ algunas veces será par. Por consiguiente, se puede definir la probabilidad de que la llave final sea 0 con el primo ρ como

$$P(k_n = 0) = P(2 \mid b_\rho) \times P(a < 2^{384}) = \frac{1}{2} \times \frac{1}{2} = \frac{1}{4} \quad (10)$$

La probabilidad no es baja y con una serie de intentos posiblemente se lograría el cometido, pero la idea fue descartada, ya que al momento de deducir el `r` este método no fue necesario.

Para [RSA](#), se intuía que el objetivo era encontrar el r que se usa para la generación de los primos. Una forma que se pensó primero era encontrarlo a partir de las siguientes ecuaciones:

$$p = a_1 r + b_1 \quad (11)$$

$$q = a_2 r + b_2 \quad (12)$$

$$n = pq = a_1 a_2 r^2 + (a_1 b_2 + a_2 b_1) r + b_1 b_2 \quad (13)$$

Sin embargo, no se logró ver forma de llegar a algo al haber demasiadas incógnitas (solo se conoce n).

La otra forma fue generar varios primos y establecer ecuaciones entre ellos para encontrar el r , pero de igual forma no se lograba llegar a algo; había muchas incógnitas y no se tenía una idea clara

para proceder. Por lo que se decidió buscar alguna pista o estrategia que se podía usar (sin ver la solución).

Se evaluó la posibilidad de usar la factorización de fermat, descrita en [17], la cual básicamente menciona que n se puede descomponer como:

$$n = \left(\frac{p+q}{2}\right)^2 - \left(\frac{p-q}{2}\right)^2 = t^2 - s^2 \quad (14)$$

ya que

$$n = \frac{p^2 + 2pq + q^2}{4} - \frac{p^2 - 2pq + q^2}{4} = \frac{p^2 + 2pq + q^2 - p^2 + 2pq - q^2}{4} = \frac{4pq}{4} = pq \quad (15)$$

Esta factorización ofrece la posibilidad de encontrar p y q si se sabe que su distancia no están grande. Si p y q son cercanos, $\frac{p-q}{2}$ sería pequeño, $\sqrt{n}$ sería un poco más pequeño que t y $t^2 - n = s^2$ sería un cuadrado perfecto, permitiendo iterar sobre

$$t = \lceil \sqrt{n} \rceil, t = \lceil \sqrt{n} \rceil + 1, t = \lceil \sqrt{n} \rceil + 2, \dots \quad (16)$$

hasta que $t^2 - n$ sea un cuadrado perfecto. Cuando se cumpla esto se tiene

$$p = t + s \wedge q = t - s \quad (17)$$

Esta estrategia no fue viable, ya que no se tenía garantía de la cercanía entre ambos primos.

En el post [13] se mencionan principalmente los temas de aproximación por fracciones continuas y el método Coppersmith. También en el repositorio [12] se menciona el uso de la matriz Lenstra-Lenstra-Lovász (LLL) para encontrar el número r . Por lo que se decidió investigar sobre la técnica LLL.

El LLL es un algoritmo de tiempo polinomial para la reducción de bases del tipo ‘lattice’ (subgrupo discreto de $\mathbb{R}^n$); el objetivo de este algoritmo es reducir la base a otra base con vectores de menor magnitud y casi-ortogonales, conservando el espacio que genera la base original. La aplicación original fue la factorización de polinomios de coeficientes racionales [22].

Por otro lado, Coppersmith derivó un algoritmo general para encontrar raíces pequeñas (en magnitud) de ecuaciones polinomiales basándose en el algoritmo LLL. Este método tiene distintas aplicaciones para vulnerar el algoritmo RSA cuando se tienen parámetros pequeños o se conoce información parcial [24].

La idea principal en este método es encontrar otro polinomio g tal que:

$$f(x) = 0 \pmod{N}, |x| < X \quad (18)$$

$$g(x) = 0, x \in \mathbb{Z} \quad (19)$$

Donde X es un límite superior.

El teorema de Howgrave-Graham establece:

Dado un polinomio $g(x)$ de una variable con n monomios y m, b enteros positivos, donde $b|N$, si:

$$g(x_0) = 0 \pmod{b^m}, |x_0| \leq X \quad (20)$$

$$\|g(xX)\| < \frac{b^m}{\sqrt{n}} \quad (21)$$

Entonces $g(x_0) = 0$, $x_0 \in \mathbb{Z}$.

Gracias a ese teorema se puede llegar al resultado:

Sea N un entero de factorización desconocida con divisor $b \geq N^\beta$, $0 < \beta \leq 1$ y $f(x)$ un polinomio mónico (primer coeficiente igual a 1) de una variable y grado δ . Entonces, para $c \geq 1$, se pueden encontrar todas las soluciones x_0 para:

$$f(x) = 0 \pmod{b}, |x_0| \leq cN^{\frac{\beta^2}{\delta}} \quad (22)$$

Uno de los casos especiales interesantes de este método es cuando $b = N$ y $c = 1$ [24]. Además, esta idea se puede extender a polinomios de más variables.

Una aplicación de Coppersmith trabaja bajo la idea de que se conoce un fragmento del mensaje que se busca descifrar.

$$c = m^e(n) \quad (23)$$

siendo m el mensaje.

$$m = m_0 + x_0 \quad (24)$$

con m_0 el fragmento de mensaje conocido. Con esto se puede plantear el polinomio

$$c \equiv (m_0 + x_0)^e(n) \quad (25)$$

$$f(x) = c - (m_0 + x)^e(n) \quad (26)$$

Con este, se puede aplicar el teorema antes mencionado y armar la matriz LLL y obtener el x . Si bien en este challenge se conoce el prefijo HTB{ y el sufijo }, no es suficiente, ya que esta aplicación necesita conocer la mayoría del mensaje y que el e sea pequeño para que la raíz x sea lo más pequeña posible [23].

Otra aplicación de Coppersmith, es Boneh-Durfee el cual se aprovecha de la característica de usar un exponente pequeño para el RSA.

$$e \times d \equiv 1 \pmod{\phi(n)} \quad (27)$$

$$e \times d = k \times \phi(n) + 1 \quad (28)$$

$$k \times \phi(n) + 1 \equiv 0 \pmod{e} \quad (29)$$

$$k \times (N + 1 - p - q) + 1 \equiv 0 \pmod{e} \quad (30)$$

$$f(x, y) = x \times (A + y) + 1 \quad (31)$$

con $A = N + 1$. Mediante ese procedimiento, se puede llegar a un polinomio y , gracias al teorema de Howgrave-Graham, se puede construir una matriz LLL. Mediante las combinaciones lineales que produce LLL se puede recuperar d . Esta aplicación fue descartada debido a que se establece que $d < n^{0.292}$ para que se pueda encontrar en un tiempo polinómico.

$$e \times d - k \times \phi(n) = 1 \quad (32)$$

$$d \approx \frac{k \times \phi(n)}{e} \quad (33)$$

Como e es 65537, d es casi del orden de n (alrededor de 2048 bits), siendo muy lento [23].

A continuación, se mencionan algunos ataques a [RSA](#) que aprovechan estos métodos bajo ciertas condiciones [24]:

- **e pequeño y mensaje m pequeño o se conoce una parte alta de los bits:** dado m^e, m' con $|m - m'| \leq N^{\frac{1}{e}}$ encontrar m .
- **Mensaje relacionados afines:** dado $c_1 = m^e \bmod N$ y $c_2 = (m + r)^e \bmod N$ con $|r| \leq N^{\frac{1}{e^2}}$ encontrar m .
- **Parte alta de bits conocida o primo pequeño:** dado $N = pq$, p' con $|p - p'| \leq N^{\frac{1}{4}}$ encontrar p .
- **Llave privada pequeña:** dado N, e con $ed \equiv 1 \bmod \phi(N)$ y $d \leq N^{0.292}$, encontrar d .

Realizando un análisis de los valores generados por el programa:

- El exponente $e = 65537$ es un número grande y bastante seguro.
- El tamaño de la llave privada d suele ser más de dos mil bits.
- La diferencia entre p y q suele ser grande, más de mil bits.
- No se conoce parte significativa del mensaje ni de los bits de p, q .

Por lo que se descartan los ataques mencionados anteriormente. Se intentó buscar una forma de aplicar el algoritmo [LLL](#) o el método de Coppersmith; sin embargo, esto no se logró pero se terminó llegando a otro método con fracciones continuas.

6. Explotación

6.1. Vulnerabilidades identificadas

ID	Nombre	Descripción
CWE-522	Insufficiently Protected Credentials	El producto transmite o guarda credenciales (o llaves) pero usa un método inseguro susceptible a la recuperación de estos.
CWE-1240	Use of a Cryptographic Primitive with a Risky Implementation	El producto implementa un algoritmo criptográfico utilizando una implementación criptográfica no estándar, no probada o no permitida/no conforme.

Cuadro 6: Principales vulnerabilidades encontradas en la fuente CWE [8]

ID	Nombre	Severidad	Descripción
CAPEC-97	Cryptanalysis	Muy alta	Encontrar vulnerabilidades en algoritmos criptográficos para descifrar un mensaje o la llave privada.

Cuadro 7: Principales patrones de ataque encontrados en la fuente CAPEC [9]

ID	Nombre	CVSS	Descripción
CVE-2017-15361	ROCA	5.9	Maneja incorrectamente la generación de claves RSA, lo que facilita que los atacantes eludan varios mecanismos de protección criptográfica mediante ataques dirigidos.
CVE-2025-65998	Apache Syncope	7.9	Cuando se configura AES, siempre se utiliza el valor de clave predeterminado, codificado directamente en el código fuente. Esto permite que un atacante malintencionado, una vez que obtenga acceso al contenido interno de la base de datos, pueda reconstruir los valores originales de la contraseña en texto plano.

Cuadro 8: Ataques relacionados encontrados en la plataforma CVE [10]

En la tabla 6 se identifican las principales vulnerabilidades relacionadas con el challenge resuelto. La vulnerabilidad CWE-522 presenta relación con el challenge, ya que la IP dada guarda un mensaje cifrado con una generación de llaves poco robusta. Del mismo modo, CWE-1240 es una vulnerabilidad explotable en la IP ya que la forma en que se generan los primos, puede terminar comprometiendo la información.

El CWE-1240 fue la vulnerabilidad que permitió resolver el challenge, ya que los algoritmos de cifrado RSA y AES son métodos que se consideran seguros; sin embargo, una insegura forma de actualizar la llave para el algoritmo AES permite fijar la misma y mediante un análisis numérico, romper a RSA, comprometiendo la información (CWE-522).

En general, el patrón de ataque que aplica en este caso es **CAPEC-97**, especificado en la tabla 7. El algoritmo **RSA** y **AES** no son los que tenían la vulnerabilidad, sino la forma en la que se generan llaves, lo que permite obtener información clave de esta y reducir el espacio de búsqueda considerablemente. Por otro lado, no se encontró un ataque específico (CVE) que represente la forma en que se resolvió este challenge, por lo que se trata más de una vulnerabilidad en un caso particular.

Respecto a la relación con los contenidos del curso, estas vulnerabilidades y patrones de ataques corresponden al reino de ‘Criptografía’ vistos en la sección ‘2.1. Taxonomía de los patrones de ataque’ relacionados con el algoritmo **RSA** y **AES**. El patrón de ataque consistió en el análisis matemático y criptográfico para obtener información del algoritmo y las llaves usadas.

6.2. Pasos para explotación

6.2.1. Paso 1

Si se generan dos primos p_1, p_2 con la opción tres del programa, se puede definir:

$$a_1 r + b_1 = p_1 \quad (34)$$

$$a_2 r + b_2 = p_2 \quad (35)$$

$$r = \frac{p_2 - b_2}{a_2} \quad (36)$$

$$a_1 \frac{p_2 - b_2}{a_2} + b_1 = p_1 \quad (37)$$

$$a_1(p_2 - b_2) + b_1 a_2 = p_1 a_2 \quad (38)$$

$$a_1 p_2 - a_1 b_2 + b_1 a_2 = p_1 a_2 \quad (39)$$

Esta ecuación se puede representar como:

$$a_1(p_2 - b_2) = a_2(p_1 - b_1) \quad (40)$$

$$\frac{a_1}{a_2} = \frac{p_1 - b_1}{p_2 - b_2} \quad (41)$$

Como b_i es pequeño en comparación con p_i (256 vs 1024 bits), se puede intuir que:

$$\frac{a_1}{a_2} \approx \frac{p_1}{p_2} \quad (42)$$

```
server_mo.py A x
62     class Server: 1 usage
88         # print("-----")
89         def aux(self): 1 usage
90             p, q= self.gen_prime(), self.gen_prime()
91             a1, b1 = p//self.r, p%self.r
92             a2, b2 = q//self.r, q%self.r
93             assert a1*self.r+b1 == p
94             assert a2*self.r+b2 == q
95             print(a1/a2, p/q)
96             print(abs(a1*q-a2*p)/(a1*q))
```

Figura 8: Función para verificar convergencia

```

3. Encrypt flag
4. Exit
> 1
It's raining primes!
a1/a2= 0.9883622832122455 p1/p2= 0.9883622832122455
dif= 4.810787443736069e-232
Choose an option:
1. Generate prime
2. Update key
3. Encrypt flag
4. Exit
> |
```

Figura 9: Diferencia entre fracciones

Para confirmar esto, se creó la función observada en la figura 8 para analizar la diferencia entre las fracciones de dos primos generados y se ejecutó localmente. En la figura 9 se muestra el resultado. La diferencia entre las fracciones es de aproximadamente 10^{-232} , lo cual se mantiene entre diferentes corridas. Esto lleva a pensar que tal vez se puede usar un método de aproximación para encontrar a_1, a_2 a partir de p_1, p_2 . Si esto es posible, se puede despejar fácilmente r , lo cual es el objetivo.

Como se mencionaba al principio, en la publicación que se consultó [13] aparecía el tema de fracciones convergentes. Este es un tema de teoría de números que permite realizar aproximaciones racionales de números reales.

Una fracción continua es un número de la forma:

$$a_0 + \frac{1}{a_1 + \frac{1}{a_2 + \frac{1}{\dots}}} \quad (43)$$

Para un número racional esta expresión es finita, mientras que para un número irracional no lo es. También se suele representar como $[a_0, a_1, \dots, a_n]$. El proceso para expandir una fracción a fracciones continuas es bastante directo y se explica más adelante.

Por otro lado, se pueden realizar aproximaciones de un número viendo a los convergentes de su fracción continua. Los convergentes de una fracción continua son fracciones continuas compuestas de una secuencia inicial de los elementos. El convergente i es:

$$C_i = a_0 + \frac{1}{a_1 + \frac{\dots}{a_{i-1} + \frac{1}{a_i}}}, \quad i \leq n \quad (44)$$

La aproximación más cercana de un número real son convergentes de su fracción continua.

Teorema: si $|R - \frac{p}{q}| \leq \frac{1}{2q^2}$ entonces $\frac{p}{q}$ es uno de los convergentes de R [26].

Por lo que en base a este teorema $\frac{a_1}{a_2}$ es un convergente de $\frac{p_1}{p_2}$ si $|\frac{p_1}{p_2} - \frac{a_1}{a_2}| \leq \frac{1}{2a_2^2}$. Como $2a_2^2 \approx 2^{769}$ por lo que $\frac{1}{2a_2^2} \approx 10^{-232}$. Esto significa que dependiendo de la corrida, es muy posible que esto se cumpla y se pueda obtener a_1, a_2 como un convergente.

```

def aux(self):
    1 usage
    a2, b2 = q//self.r, q%self.r
    assert a1*self.r+b1 == p
    assert a2*self.r+b2 == q
    assert (a1 * q - a2 * p) / (a2 * q) <= 1 / (2 * (a2 ** 2))
    print(f"a1/a2= {a1/a2} p1/p2= {p/q}")
    print("dif= ", abs(a1*q-a2*p)/(a1*q))

```

server_mo x ag A x

```

:
3. Enter prime key
4. Exit
> 1
It's raining primes!
a1/a2= 1.3761868901461118 p1/p2= 1.3761868901461118
dif= 3.385498855932588e-232
Choose an option:
1. Generate prime
2. Update key
3. Encrypt flag

```

Figura 10: Verificación del teorema de convergentes

En la figura 10 se muestra que se añade a la función `aux` el `assert` del teorema y el programa corre correctamente.

6.2.2. Paso 2

Al obtener a_1 y a_2 , es posible calcular b_1 si $b_1 < a_1$:

$$p_1 = a_1 r + b_1 \quad (45)$$

$$b_1 = p_1 \bmod a_1 \quad (46)$$

Ahora con b_1 se puede calcular r mediante

$$p_1 = a_1 r + b_1 \quad (47)$$

$$r = \frac{p_1 - b_1}{a_1} \quad (48)$$

6.2.3. Paso 3

Una vez obtenido el r correcto, el objetivo es factorizar la llave pública n que usa para el cifrado con [RSA](#). Para esto se analiza:

$$p = a_p r + b_p \quad (49)$$

$$q = a_q r + b_q \quad (50)$$

$$n = pq = (a_p r + b_p)(a_q r + b_q) \quad (51)$$

$$n = a_p a_q r^2 + (a_p b_q + a_q b_p) r + b_p b_q \quad (52)$$

El objetivo es encontrar (a_p, b_p) o (a_q, b_q) . Como se conocen r y n , es posible sacar:

$$\begin{aligned} z &= n \bmod r = b_p b_q \\ y &= \frac{n - z}{r} \bmod r = a_p b_q + a_q b_p \text{ si } a_p b_q + a_q b_p < r \\ x &= \frac{n - yr - z}{r^2} = a_p a_q \end{aligned} \quad (53)$$

No se logró obtener los valores buscados por medios de despejes en base a las ecuaciones anteriores, por lo que se tuvo que plantear de otra forma.

$$u = a_p b_q \quad (54)$$

$$v = a_q b_p \quad (55)$$

$$y = u + v \quad (56)$$

$$uv = xz \quad (57)$$

Si se logra encontrar u, v es posible encontrar (a_p, b_p) . Como los primos son de la forma $F = ar + b$, entonces necesariamente se cumple que el máximo común divisor (mcd) $\text{mcd}(a, b) = 1$ (si $\text{mcd}(a, b) = d$ entonces $d|F$), por lo que:

$$\text{mcd}(u, x) = \text{mcd}(a_p b_q, a_p a_q) = a_p \quad (58)$$

$$\text{mcd}(v, z) = \text{mcd}(a_q b_p, b_p b_q) = b_p \quad (59)$$

Para encontrar u, v se puede plantear el siguiente polinomio:

$$(T - u)(T - v) = T^2 - (u + v)T + uv = T^2 - yT + xz \quad (60)$$

Como se trata de una ecuación cuadrática y sus raíces corresponden a u, v , se pueden encontrar estas por medio del discriminante:

$$a = 1, \quad b = -y, \quad c = xz \quad (61)$$

$$\Delta = b^2 - 4ac = y^2 - 4xz = u^2 - 2uv + v^2 = (u - v)^2 \quad (62)$$

Por lo que el discriminante siempre debería ser positivo y tener raíz entera $\sqrt{\Delta} = |u - v|$.

$$u = \frac{-b + \sqrt{\Delta}}{2a} = \frac{y + \sqrt{\Delta}}{2} \quad (63)$$

$$v = \frac{-b - \sqrt{\Delta}}{2a} = \frac{y - \sqrt{\Delta}}{2} \quad (64)$$

De esta forma $a_1 = \text{mcd}(u, x)$ y $b_1 = \text{mcd}(v, x)$:

$$p = a_1 r + b_1 \quad (65)$$

$$q = \frac{n}{p} \quad (66)$$

6.2.4. Paso 4

Una vez obtenidos p, q , calcular la llave privada es directo:

$$\phi(n) = (p - 1)(q - 1) \quad (67)$$

$$ed \equiv 1 \mod \phi(n) \quad (68)$$

$$d = e^{-1} \mod \phi(n) \quad (69)$$

Finalmente, el mensaje cifrado m con [RSA](#) corresponde a $c^d \mod n$.

6.2.5. Paso 5

Con el valor de r , igualar la llave usada en el cifrado [AES](#) a 0 se puede hacer con facilidad, ya que como se demostró en la sección [5.3](#), se puede obtener

$$k_{n,i} = (0 + 2 \times b_i + k_{a,i}) \times k_{1,i} (r) \quad (70)$$

donde $k_{a,i}$ es la i -ésima posición de la antigua llave, $k_{1,i}$ la i -ésima posición del arreglo otorgado por el usuario y $k_{n,i}$ la i -ésima posición del arreglo que representa la llave final. Si se genera un arreglo de tamaño 256, donde cada elemento es igual a $r + 2$ y se utiliza la segunda opción del menú de `server.py`, se obtendrá

$$k_{n,i} = (0 + 2 \times b_i + k_{a,i}) \times (r + 2) (r) \quad (71)$$

$$k_{n,i} = (0 + 2 \times b_i + k_{a,i}) \times 2 (r) \quad (72)$$

garantizando que el valor final, tras aplicar módulo 2, sea 0 para la nueva llave. Con todo esto, es posible realizar la explotación y obtener el mensaje.

6.2.6. Implementación

La solución se implementó en el archivo `ag.py`, donde se muestran las funciones para los cálculos antes definidos.

```

1  def continued_fraction(n, d): 1usage
2      cf = []
3      while d:
4          a = n // d
5          cf.append(a)
6          n, d = d, n - a * d
7      return cf
8
9  def convergents(cf : {__iter__}): 1usage
10     p0, p1 = 0, 1
11     q0, q1 = 1, 0
12     for a in cf:
13         p2 = a * p1 + p0
14         q2 = a * q1 + q0
15         yield p2, q2
16         p0, p1 = p1, p2
17         q0, q1 = q1, q2,
18
19  def find_rational_approx(p : {__mod__, __sub__}, q : {__mod__, __sub__}): 1usage
20     cf = continued_fraction(p, q)
21     for x, y in convergents(cf):
22         z, w = x.bit_length(), y.bit_length()
23         if max(z, w) < 385 and min(z, w) >= 380 and test(p, q, x, y):
24             return True
25     return False

```

Figura 11: Fragmento de código donde se calcula la fracción continuas y sus convergentes

La primer función definida en la figura 11, muestra el cálculo de la fracción continua de $\frac{p}{q}$. Se itera hasta que el divisor sea 0 guardando la parte entera (líneas 2-4) En la línea 5, se asignar al dividendo el divisor y al divisor, el resto. La razón es que al momento de calcular la fracción continua, se tiene

$$\frac{n}{d} = a + \frac{n - a}{d} = a + \frac{1}{\frac{d}{n-a}} \quad (73)$$

donde $a = \lfloor \frac{n}{d} \rfloor$.

La segunda función definida calcula los diferentes convergentes de la función continua. En las líneas 10 y 11, se definen los casos base para después en las líneas 12 a la 17, se realiza un proceso iterativo que calcula una recursión para obtener todos los convergentes, y usar la palabra reservada “yield” para pausar la función y procesar una pareja candidata. La recursión planteada es de la forma

$$p_i = a \times p_{i-1} + p_{i-2} \quad (74)$$

$$q_i = a \times q_{i-1} + q_{i-2} \quad (75)$$

donde a es unas de las partes enteras de la fracción continua de la función `continued_fraction()` y, los p y q los estados anteriores del numerador y denominador. Dicha recursión nace desde el análisis

del comportamiento del cálculo de los convergentes. Se sabe que el primer convergente $C_0 = \frac{a_0}{1}$ por lo que se podría definir

$$p_0 = a_0 \quad (76)$$

$$q_0 = 1 \quad (77)$$

ahora bien, el C_1 es

$$C_1 = a_0 + \frac{1}{a_1} \quad (78)$$

$$C_1 = \frac{a_0 \times a_1 + 1}{a_1} \quad (79)$$

por lo tanto

$$p_1 = a_0 \times a_1 + 1 \quad (80)$$

$$q_1 = a_1 \quad (81)$$

del mismo modo, C_2 es

$$C_2 = a_0 + \frac{1}{a_1 + \frac{1}{a_2}} \quad (82)$$

$$C_2 = a_0 + \frac{1}{\frac{a_1 \times a_2 + 1}{a_2}} \quad (83)$$

$$C_2 = a_0 + \frac{a_2}{a_1 \times a_2 + 1} \quad (84)$$

$$C_2 = \frac{a_0 \times a_1 \times a_2 + a_0 + a_2}{a_1 \times a_2 + 1} \quad (85)$$

entonces

$$p_2 = a_0 \times a_1 \times a_2 + a_0 + a_2 \quad (86)$$

$$q_2 = a_1 \times a_2 + 1 \quad (87)$$

Si se realiza un poco de trabajo algebraico sobre p_2 , se obtiene

$$p_2 = a_2 \times (a_0 \times a_1 + 1) + a_0 \quad (88)$$

$$p_2 = a_2 \times p_1 + p_0 \quad (89)$$

de la misma forma con q_2

$$q_2 = a_2 \times q_1 + q_0 \quad (90)$$

En la línea 19 se define la función `find_rational_approx()`, la cual itera sobre todos los posibles convergentes y verificar que la longitud de ambos es coherente con lo esperado y se prueban en la función `test()` (líneas 20-23), que se define en la figura 12. Si la pareja, cumple todas las condiciones se termina el proceso retornando `True`, caso contrario, se retorna `False`.

```

27 def test(p : {__mod__, __sub__}, q : {__mod__, __sub__}, a1 : {__mul__}, a2 : {__mul__}):
28     b1, b2 = p % a1, q % a2
29     r1, r2 = (p - b1) // a1, (q - b2) // a2
30     from Crypto.Util.number import isPrime
31     if r1 != r2 or not isPrime(r1): return False
32     assert a1 * r1 + b1 == p and a2 * r2 + b2 == q
33     calc_inverse(r1)
34     return True
35
36 def calc_inverse(r : {__mul__}): 1usage
37     global D, R
38     R = r
39     n = N
40     c = n % r
41     b = ((n - c) // r) % r
42     a = (n - c - b * r) // (r * r)
43     assert a * (r * r) + b * r + c == n
44     from math import isqrt
45     disc = abs(b * b - 4 * a * c)
46     k = isqrt(disc)
47     assert k * k == disc
48     x = (b + k) // 2
49     y = (b - k) // 2
50     from Crypto.Util.number import GCD
51     a1 = GCD(x, a)
52     b1 = GCD(y, c)
53     q = a1 * r + b1
54     assert n % q == 0
55     p = n // q
56     phi = (p - 1) * (q - 1)
57     from Crypto.Util.number import inverse
58     d = inverse(E, phi)
59     D = d

```

Figura 12: Fragmento de código para verificar la pareja de convergentes y calcular el inverso del [RSA](#)

La función `test()`, en la figura 12, obtiene los b 's a partir del módulo de la fracción convergente (línea 28). Con estos b , se calcula un r por cada primo (línea 29). En la línea 30 se importa la función `isPrime()` y se utiliza para verificar que r es primo y que ambos r 's son iguales, y que el r puede volver a generar ambos primos (línea 31 y 32). Al cumplir con estas condiciones, se procede a calcular el inverso (línea 33).

La función `calc_inverse()`, calcula el inverso del [RSA](#). En las líneas 40, 41 y 42, se calculan los valores de x , y y z definidos en la ecuación 53, y en la línea 43 se verifica su coherencia, es decir, que todavía se cumplan con la ecuación 52. Esta verificación es importante, ya que se sabe que los b pueden ser de hasta 256 bits y el producto de ambos, módulo r siempre es $b_p b_q$ ($256 * 2 = 512 < 640$, r tiene 640 bits), pero con y no ocurre lo mismo. En el peor caso, $a_p b_q$ y $a_q b_p$ tiene $384 + 256 = 640$

bits y la suma de ambos puede superar esa cantidad, por lo tanto puede que al aplicar módulo r , el valor no sea el deseado (Cabe recalcar, que durante las pruebas hechas, ese `assert` nunca falló, pero la posibilidad si existe).

En la línea 44, se importa la función que calcula la raíz entera, se calcula el discriminante de la cuadrática planteada en la ecuación 60, se calcula la raíz y se verifica que el cuadrado de la raíz corresponda al discriminante (líneas 45-47). En las líneas 48 y 49, se extraen las raíces de la cuadrática y se importa una función que calcula el “mcd” para números de gran tamaño (línea 50). En las líneas 51 y 52, se obtienen a_1 y b_1 , respectivamente, para su posterior uso calculando un primo q (línea 53), se verifica la existencia del otro primo p (línea 54) y se calcula p (línea 55). Ahora con ambos primos, se calcula ϕ (línea 56) y se calcula el inverso del RSA (líneas 57-59), usando el inverso de `Crypto`.

```

66     def construct_key(): 1usage
67         return str([R + 2] * 256)
68
69     def n2b(n: int, k: int = 32) -> bytes: return n.to_bytes(k, byteorder: 'big')
70
71     def decrypt(): 1usage
72         print("---> Vamos a descifrar")
73         msg = pow(MSG, D, N)
74         print("Descifrado RSA: ", msg)
75         from Crypto.Cipher import AES
76         from Crypto.Util.Padding import unpad
77         from Crypto.Util.number import long_to_bytes
78         cipher = AES.new(n2b(0), AES.MODE_ECB)
79         msg = unpad(cipher.decrypt(long_to_bytes(msg)), AES.block_size)
80         print("Descifrado AES: ", msg)
81
82
83     N = 0
84     E = 65537
85     R = 0
86     D = 0
87     MSG = 0

```

Figura 13: Funciones para romper AES

En la figura 13, se define la función `construct_key()`, la cual genera un arreglo de tamaño 256 con todos los elementos siendo $r + 2$. Seguido, en la línea 69, se define la misma función otorgada en el archivo `server.py` y explicada en la tabla 4.2.

En la línea 71 se define la función `decrypt()`, la cual le aplica la inversa al mensaje cifrado con RSA e importa AES, la función para quitar el padding y para convertir un número grande a bytes (líneas 73-77). En la línea 78, se crea una instancia de AES y se descifra el mensaje en la línea 79. Además, en las línea 83-87, se definen ciertas variables globales de RSA.

```

89     def attack():
90         global N, MSG
91         from pwn import remote
92         io = remote(host="154.57.164.65", port=31146)
93         print("---> Pidiendo flag cifrado")
94         io.sendlineafter(delim: b"> ", data: b"3")
95         io.recvuntil(b"flag:\n")
96         N, _, MSG = eval(io.recvline().decode().strip())
97         print("N= ", N)
98         print("C= ", MSG)
99         while 1:
100             print("---> Probando pareja")
101             io.sendlineafter(delim: b"> ", data: b"1")
102             io.recvuntil(b"primes!\n")
103             p = int(io.recvline().strip())
104             print("P2 ", p)
105             io.sendlineafter(delim: b"> ", data: b"1")
106             io.recvuntil(b"primes!\n")
107             q = int(io.recvline().strip())
108             print("Q2 ", q)
109             if find_rational_approx(p, q): break
110             print("---> Actualizando llave de AES")
111             io.sendlineafter(delim: b"> ", data: b"2")
112             io.sendlineafter(delim: b"key:\n", construct_key().encode())
113             print("---> Pidiendo el nuevo mensaje cifrado")
114             io.sendlineafter(delim: b"> ", data: b"3")
115             io.recvuntil(b"flag:\n")
116             _, _, MSG = eval(io.recvline().decode().strip())
117             print("C= ", MSG)
118             io.close()
119             decrypt()

```

Figura 14: Función que orquesta el ataque

La función `attack()` presentada en la figura 14, se basa en una función presentada en [12] (solo para la parte de IO). En resumen, se establece la conexión con una IP y puerto determinado (líneas 91 y 92). Posteriormente, se solicita y obtiene el mensaje cifrado, el n y el e (líneas 94-96). Seguido, en la línea 99, se consultan dos primos (líneas 101 - 107) y se trata de encontrar un r válido (línea 109). Esto se hace hasta encontrar un par válido o no se cumpla algún `assert`, con las pruebas realizadas, a lo mucho se consultaban 5 pares de números primos. Seguido, se actualiza la llave AES (líneas 111 y 112), se solicita el mensaje cifrado con la nueva llave (líneas 114-116), se cierra la comunicación (línea 118) y se descifra el mensaje (línea 119).

6.2.7. Demostración

Primero se debe encender la maquina en [HTB](#), figura 15, para poder obtener la IP. Seguido es necesario, conectarse a la VPN de [HTB](#), mediante el comando `sudo openvpn pack.ovpn`, donde `pack.ovpn` es el archivo que otorga [HTB](#) para establecer conexión con la VPN, figura 16.

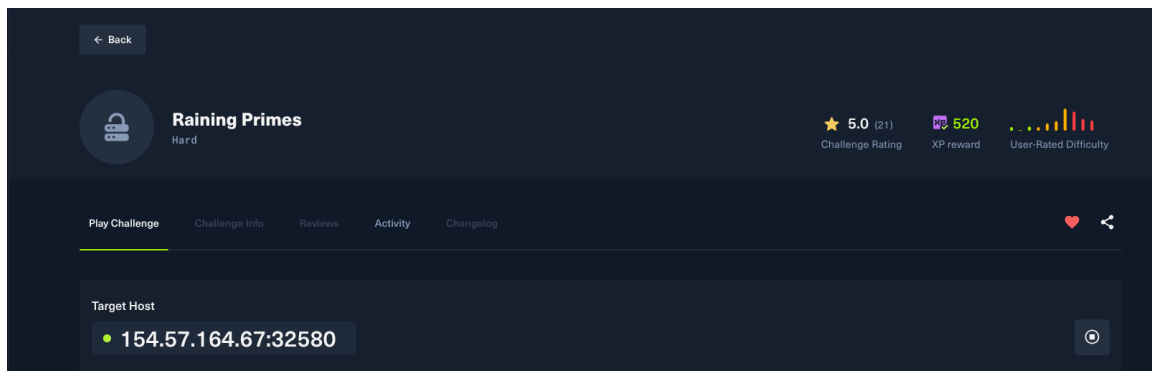


Figura 15: Encender la maquina para realizar la conexión

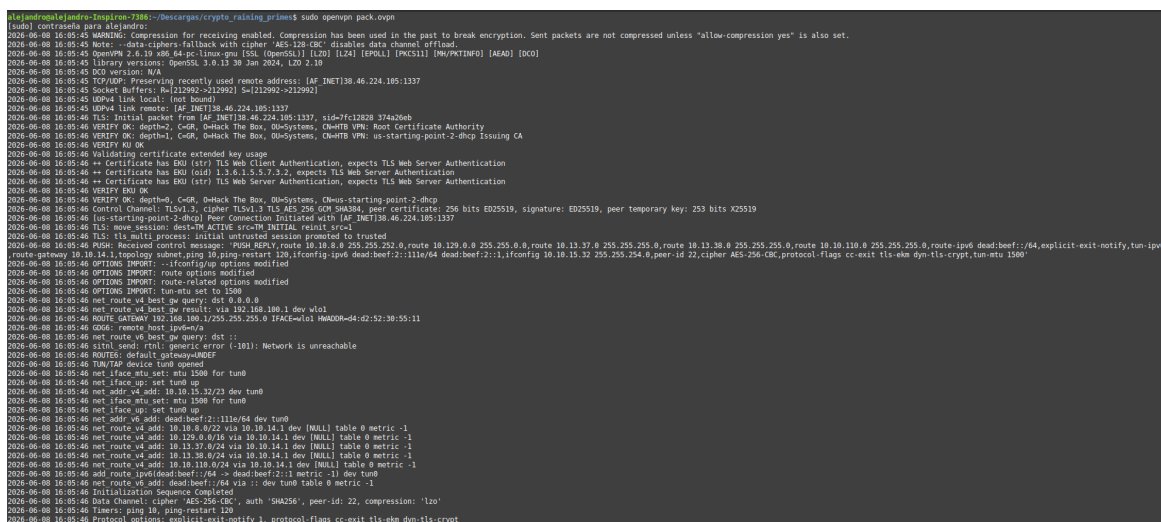


Figura 16: Establecimiento de la conexión con la VPN

Una vez con el ambiente establecido y con la IP, se debe agregar la IP al archivo `ag.py`, donde se muestra en la figura 14. Ahora, se compila el archivo con `python3 ag.py` y se establece conexión, figura 17.

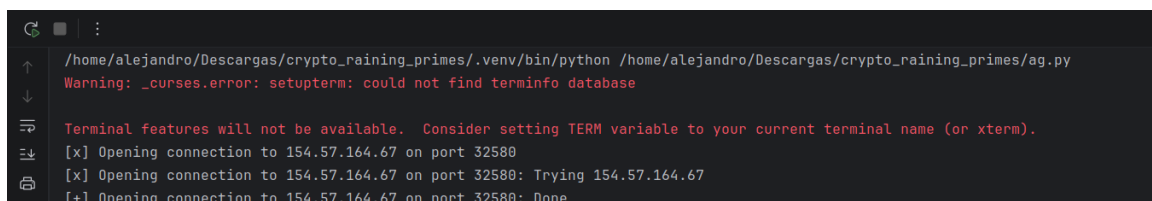


Figura 17: Establecimiento de la conexión con la IP

Una vez con la conexión, se obtiene el mensaje cifrado y el n del [RSA](#), figura 18.

```
Terminal features will not be available. Consider setting TERM variable to your current terminal name (or xterm).
[x] Opening connection to 154.57.164.67 on port 32580
[x] Opening connection to 154.57.164.67 on port 32580: Trying 154.57.164.67
[+] Opening connection to 154.57.164.67 on port 32580: Done
----> Pidiendo flag cifrado
N= 873021564784153142591634031576372866466156469413078918411598635562312293508393836860681335988827920630225777182208341311192062568586660714381658399942833976291687046880384
C= 18330259608426366826679492007281165838502911105212775505648299450234079926590499247064783498161675885972352339247989292302481740347797285136746376967055700119675473684662
```

Figura 18: Obtención del N y del c

Después se obtiene dos primos para intentar extraer r de ellos, figura 19.

```
/home/alejandra/Descargas/crypto_raining_primes/.venv/bin/python /home/alejandra/Descargas/crypto_raining_primes/ag.py
Warning: _curses.error: setupterm: could not find terminfo database

Terminal features will not be available. Consider setting TERM variable to your current terminal name (or xterm).
[x] Opening connection to 154.57.164.67 on port 32580
[x] Opening connection to 154.57.164.67 on port 32580: Trying 154.57.164.67
[+] Opening connection to 154.57.164.67 on port 32580: Done
----> Pidiendo flag cifrado
N= 873021564784153142591634031576372866466156469413078918411598635562312293508393836860681335988827920630225777182208341311192062568586660714381658399942833976291687046880384
C= 18330259608426366826679492007281165838502911105212775505648299450234079926590499247064783498161675885972352339247989292302481740347797285136746376967055700119675473684662
----> Probando pareja
P2 9138085430645917247416558170256503537013366819067951511680052856992170010591168936164434675516184930752516407268554131460488859892927186095185548655308482525090979263076
Q2 93941534743715504725366995780126037121820131474956240895772895262914856218393045633544895799312920821059847394442131591480010481518026598344447651205083045361714780742580
```

Figura 19: Probar la pareja de primos generados

Se puede observar en la figura 20, que la extracción de los valores relacionados con [RSA](#), fue un éxito.

```
/home/alejandra/Descargas/crypto_raining_primes/.venv/bin/python /home/alejandra/Descargas/crypto_raining_primes/ag.py
Warning: _curses.error: setupterm: could not find terminfo database

Terminal features will not be available. Consider setting TERM variable to your current terminal name (or xterm).
[x] Opening connection to 154.57.164.67 on port 32580
[x] Opening connection to 154.57.164.67 on port 32580: Trying 154.57.164.67
[+] Opening connection to 154.57.164.67 on port 32580: Done
----> Pidiendo flag cifrado
N= 873021564784153142591634031576372866466156469413078918411598635562312293508393836860681335988827920630225777182208341311192062568586660714381658399942833976291687046880384
C= 18330259608426366826679492007281165838502911105212775505648299450234079926590499247064783498161675885972352339247989292302481740347797285136746376967055700119675473684662
----> Probando pareja
P2 9138085430645917247416558170256503537013366819067951511680052856992170010591168936164434675516184930752516407268554131460488859892927186095185548655308482525090979263076
Q2 93941534743715504725366995780126037121820131474956240895772895262914856218393045633544895799312920821059847394442131591480010481518026598344447651205083045361714780742580
----> Buen intento
r= 291517871002215376892374577023552661611195546424948569338501755821325991088150047019851578995793046338123384211049505705311817933385224329999122742942522772767267177337246
P= 9202275665100397238284643931550775287935194243767069808372269146752648718817395653033494076306008312590844301547296982121157349803067197534806570240787518405929157770853
Q= 9487018174157559736059239106866188892065473648128581984672579869038718989745472966590814240212598803416854389273670723552484783511146319409668339013850518707716120463996
d= 48181611120663434995703265221889484022895682962870800907834446561639361084298627790508555195945809846528378517846981861708613078396143152549069136150199796464019532204707
```

Figura 20: r , P , Q y d deducidos de las funciones antes definidas

En la figura 21, se actualiza la llave [AES](#) y se solicita el mensaje nuevamente y en la figura 22, se descifró el mensaje exitosamente.

```

/home/alejandra/Descargas/crypto_raining_primes/.venv/bin/python /home/alejandra/Descargas/crypto_raining_primes/ag.py
Warning: _curses.error: setupterm: could not find terminfo database

Terminal features will not be available. Consider setting TERM variable to your current terminal name (or xterm).
[x] Opening connection to 154.57.164.67 on port 32580
[x] Opening connection to 154.57.164.67 on port 32580: Trying 154.57.164.67
[+] Opening connection to 154.57.164.67 on port 32580: Done
----> Pidiendo flag cifrado
N= 87302156478415314259163403157637286466156469413078918411598635562312293508393836860681335988827920630225777182208341311192062568586660714381658399942833976291687046880384
C= 183302596084263668266794920872811658385029111052127750564829945023407992659049924706478349816167588597235233924798929230248174034779728513674637667055700119675473684662
----> Probando pareja
P2 91380854306459172474165581702560353701336681906795151511680052856992170010591168936164443467551618493075251640726855413146048859892927186095185548655308482525090979263076
Q2 939415347437155047253669995780126037121820131474956240895772895262914856218393045633544895799312920821059847394442131591480010481518026598344447651205083045361714780742580
----> Buen intento
r= 2915178710022153768923745770235526616111955466249485693385017558213259910881500470198515789579304633812338421106950570531181793338522432999122762942522772767267173337246
P= 9202275665100397238284643931550775287935194263767069808372269146752648718817395653033494076306000312590844301547296982121157349803067197534806570240787518405929157770853
Q= 948701817415755973605923910686618889206547364812858198467257986903871898974547296659001424021259880344168543892736707235524847835111463194096683390138505187077716120463996
d= 40181611120663434995703265221889484022895682962870009078344656163936108429862779050855519594580984465283785178469018617086130783961431525490691361501979796464019532204707
----> Actualizando llave de AES
----> Pidiendo el nuevo mensaje cifrado
C= 60126167815489150164075918677859725803936280681509608324115601526493888079635817076920291283626088991738928727142436723020942436226272888217342976620121369910217670213142
[*] Closed connection to 154.57.164.67 port 32580

```

Figura 21: Actualización del AES y obtención del mensaje con el nuevo cifrado

```

/home/alejandra/Descargas/crypto_raining_primes/.venv/bin/python /home/alejandra/Descargas/crypto_raining_primes/ag.py
Warning: _curses.error: setupterm: could not find terminfo database

Terminal features will not be available. Consider setting TERM variable to your current terminal name (or xterm).
[x] Opening connection to 154.57.164.67 on port 32580
[x] Opening connection to 154.57.164.67 on port 32580: Trying 154.57.164.67
[+] Opening connection to 154.57.164.67 on port 32580: Done
----> Pidiendo flag cifrado
N= 87302156478415314259163403157637286466156469413078918411598635562312293508393836860681335988827920630225777182208341311192062568586660714381658399942833976291687046880384
C= 183302596084263668266794920872811658385029111052127750564829945023407992659049924706478349816167588597235233924798929230248174034779728513674637667055700119675473684662
----> Probando pareja
P2 91380854306459172474165581702560353701336681906795151511680052856992170010591168936164443467551618493075251640726855413146048859892927186095185548655308482525090979263076
Q2 939415347437155047253669995780126037121820131474956240895772895262914856218393045633544895799312920821059847394442131591480010481518026598344447651205083045361714780742580
----> Buen intento
r= 2915178710022153768923745770235526616111955466249485693385017558213259910881500470198515789579304633812338421106950570531181793338522432999122762942522772767267173337246
P= 9202275665100397238284643931550775287935194263767069808372269146752648718817395653033494076306000312590844301547296982121157349803067197534806570240787518405929157770853
Q= 948701817415755973605923910686618889206547364812858198467257986903871898974547296659001424021259880344168543892736707235524847835111463194096683390138505187077716120463996
d= 40181611120663434995703265221889484022895682962870009078344656163936108429862779050855519594580984465283785178469018617086130783961431525490691361501979796464019532204707
----> Actualizando llave de AES
----> Pidiendo el nuevo mensaje cifrado
C= 60126167815489150164075918677859725803936280681509608324115601526493888079635817076920291283626088991738928727142436723020942436226272888217342976620121369910217670213142
[*] Closed connection to 154.57.164.67 port 32580
----> Vamos a descifrar
Descifrado RSA: 3871417465618336060085354331214772075306064438484666113222477081588646395609020875178695640272728291530299528948844
Descifrado AES: b'HTB{5245e6e56e5293af7b818f3f8273a8e1d}'

```

Figura 22: Mensaje descifrado

6.2.8. Aspectos adicionales

El `assert` de la línea 47 de la figura 12, en ocasiones suele fallar, como se muestra en la figura 23. El motivo real de esto se desconoce, ya que como se demostró en el paso 3, sección 6.2.3, el discriminante siempre es positivo, por lo cual siempre deberían de existir las soluciones. La primera hipótesis sobre la causa del fallo es que alguna de las operaciones ejecutadas en las líneas anteriores produce un resultado incorrecto. Sin embargo, ninguno de los `assert` incorporados en dicha función falla durante la ejecución, lo que sugiere que las operaciones verificadas se están realizando correctamente y que el origen del problema no se encuentra en ese punto. La segunda hipótesis es que pueda existir más de un valor de r que satisfaga las condiciones iniciales descritas en el Paso 3 (6.2.3). No obstante, únicamente uno de esos valores cumple además la condición impuesta por el discriminante, lo que podría explicar el error. En general, los errores se pueden producir porque no se cumple alguna condición, pero la probabilidad es bastante alta de que el programa realice el ataque correctamente. Haciendo algunas pruebas, de seis intentos se lograron tres, así tiende a funcionar la mitad de las veces.

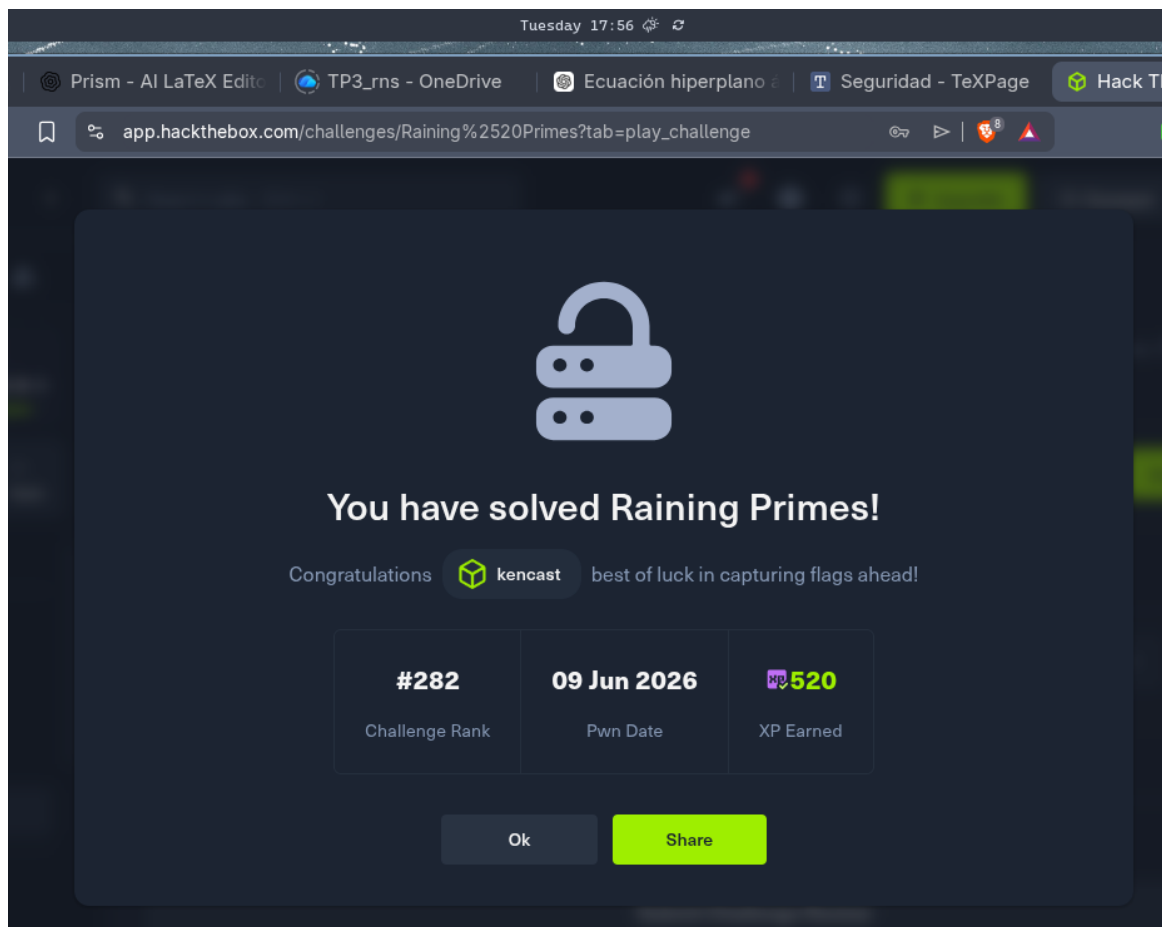


Figura 25: Resolución del challenge

8. Remediación

En este documento se presentan dos vulnerabilidades, una en la implementación del algoritmo [AES](#) y otra en la implementación del [RSA](#). Sin embargo, la primera no puede ser explotada de forma que comprometa información, sin la segunda. Primero, si se requiere implementar un sistema de cambio de la llave [AES](#), sería ideal que esta sea generada completamente aleatoria, sin intervención de una llave otorgada por el atacante. Se puede usar funciones como `randrange()` o `randint()` de la biblioteca de `Crypto`. Si se opta por conservar un mecanismo en el que el usuario participe en la generación de la llave, debe emplearse un algoritmo criptográficamente seguro que impida la predicción de la llave resultante, independientemente del grado de influencia que el usuario tenga sobre los parámetros de entrada. Por ejemplo, en vez de aplicar módulo r se podría aplicar un módulo un número aleatorio.

En el caso del [RSA](#), la vulnerabilidad nace producto de que sus primos comparten un término en común, característica que atrae bastante la atención. El principal consejo es generar ambos primos con una función como `getPrime()` de `Crypto`, de tal forma que no se pueda establecer una relación directa entre ellos; los números primos de la llave pública tienen que ser independientes y aleatorios. En esta ocasión, la posibilidad de mantener una forma poco convencional de generar los primos de [RSA](#), no es recomendable bajo ninguna circunstancia, ya que este tipo de métodos suele revelar información relevante para un atacante. En el peor de los casos, dicha información podría permitir reducir

el problema original a otro ya conocido o resoluble (véase la sección 5.3), aprovechando propiedades matemáticas que faciliten significativamente el ataque y comprometan la seguridad del sistema.

Por otro lado, es necesario verificar que las claves que se generan para estos algoritmos sean seguras y lo suficientemente grandes, así como aleatorias. Para este caso, la biblioteca **Crypto** fue una muy buena opción y el uso de [RSA-2048](#) y [AES-256](#).

Por último, es necesario tener cuidado con las diferencias de tamaños (principalmente en bits) entre los números que se usan, debido a que diferencias marcadas pueden introducir relaciones o efectos no deseados. En este caso, la diferencia de bits entre los números `get_prime()` y `b()` (1024 vs 256 bits) permite la aproximación por fracciones continuas. Es importante que los números tengan una cantidad de bits similar, pero que tampoco sean muy cercanos entre sí.

Las buenas prácticas relacionadas con los principios de seguridad desde el diseño son las siguientes [11]:

- **Definir y usar estándares criptográficos:** es crítico usar los mecanismos criptográficos de forma correcta y segura, entendiendo cómo estos funcionan y definiendo las formas de proteger y manejar las llaves de cifrado y certificados. Esto incluye también usar bibliotecas que tengan una seguridad comprobada para los procedimientos criptográficos y matemáticos.
- **Requerir el uso de funciones y herramientas con seguridad comprobada:** asegurarse de que los equipos de desarrollo usen soluciones bien establecidas y de seguridad comprobada. Manteniéndose al día con los cambios y las versiones de las herramientas. Por ejemplo, usando una versión segura de RSA.

9. Lecciones aprendidas

Con este challenge se pudo profundizar y lograr una buena comprensión de los algoritmos [RSA](#) y [AES](#), así como de las vulnerabilidades que estos pueden tener si no se implementan de forma correcta. Además, se estudió sobre los casos “simples” de [RSA](#) y los diferentes algoritmos de ataques sobre estos como es el caso de Coopersmith y sus aplicaciones como Boneh-Durfee. Del mismo modo, se aprendió sobre la reducción [LLL](#), la factorización de Fermat y fracciones continuas.

En general, se pudo observar e indagar un poco sobre los métodos criptográficos y prácticas que se usan para una transmisión segura de información, en especial cómo se combinan los algoritmos de cifrado simétricos y asimétricos para transmitir información de forma segura y eficiente. Adicionalmente, se estudió la biblioteca **Crypto** y sus diferentes funciones, la cual que ofrece poderosas funciones ideales para el cifrado de información. Lo que se haría diferente en este challenge sería notar de forma más directa algunas propiedades matemáticas que permitían descifrar el mensaje.

10. Acrónimos

[RSA](#) Rivest, Shamir y Adleman

[AES](#) Advanced Encryption Standard

[HTB](#) Hack The Box

[LLL](#) Lenstra-Lenstra-Lovász

11. Recursos consultados

- [1] Información sobre el algoritmo RSA: https://en.wikipedia.org/wiki/RSA_cryptosystem
- [2] Módulo de Python usado en el script: <https://www.pycrypto.org/api/current/Crypto.Util.number-module.html>
- [3] Explicación del algoritmo RSA: <https://www.geeksforgeeks.org/computer-networks/rsa-algorithm-cryptography/>
- [4] Explicación del algoritmo AES: https://es.wikipedia.org/wiki/Advanced_Encryption_Standard
- [5] Funcionamiento de la biblioteca de AES <https://pycryptodome.readthedocs.io/en/latest/src/cipher/aes.html>
- [6] Explicación de las funciones de la biblioteca Crypto Util, como padding y long to bytes. <https://pycryptodome.readthedocs.io/en/latest/src/util/util.html>
- [7] Funcionamiento de pow en python https://www.w3schools.com/python/ref_func_pow.asp
- [8] Common Weakness Enumeration: <https://cwe.mitre.org/index.html>
- [9] Common Attack Pattern Enumeration and Clasification: <https://capec.mitre.org/index.html>
- [10] Common Vulnerabilities and Exposures: <https://www.cve.org/About/Overview>
- [11] Principios de seguridad del software desde el diseño: <https://www.microsoft.com/en-us/securityengineering/sdl/practices>
- [12] Github con una solución diferente <https://github.com/Martes678/HTB-challenges/tree/main/Raining%20Primes>
- [13] Post de Linkedin con ideas generales del problema https://www.linkedin.com/posts/trefor-benbow-6264589a_owned-raining-primes-from-hack-the-box-activity-7379932889519497216
- [14] Información sobre la función randbits <https://www.analyticslane.com/2023/02/27/generacion-de-numeros-aleatorios-criptograficamente-seguros-en-python/>
- [15] Información de la función to-bytes <https://www.geeksforgeeks.org/python/to-bytes-in-python/>
- [16] Información de la función literal-eval <https://www.geeksforgeeks.org/python/difference-between-eval-and-ast-literal-eval-in-python/>
- [17] Explición de técnicas para vulnerar RSA <https://www.youtube.com/watch?v=C6abHMw8uoo>
- [18] Explicación de la reducción LLL <https://www.lattices.io/algorithms/lll>
- [19] Explicación del ataque Coppersmith https://en.wikipedia.org/wiki/Coppersmith%27s_attack

- [20] Github con explicación e implementación de Coppersmith y Boneh-Durfee <https://github.com/mimoo/RSA-and-LLL-attacks>
- [21] Información de la función raíz entera https://www.w3schools.com/python/ref_math_isqrt.asp
- [22] Explicación de LLL https://en.wikipedia.org/wiki/Lenstra%E2%80%93Lenstra%E2%80%93Lov%C3%A1sz_lattice_basis_reduction_algorithm
- [23] Explicación de Coppersmith y Boneh-Durfee <https://www.youtube.com/watch?v=3cicTG3zeVQ>
- [24] Paper con explicación y aplicaciones de LLL y Coppersmith. https://www.researchgate.net/publication/227200196_Using_LLL-Reduction_for_Solving_RSA_and_Factorization_Problems
- [25] Función from_bytes <https://docs.python.org/3/library/stdtypes.html>
- [26] Explicación y formulas sobre fracciones continua <https://math.mit.edu/classes/18.095/2024IAP/L5N.pdf>